SOVEREIGN SHARDS — STATE ARCHITECTURE MANIFEST

Project: sovereign-shards

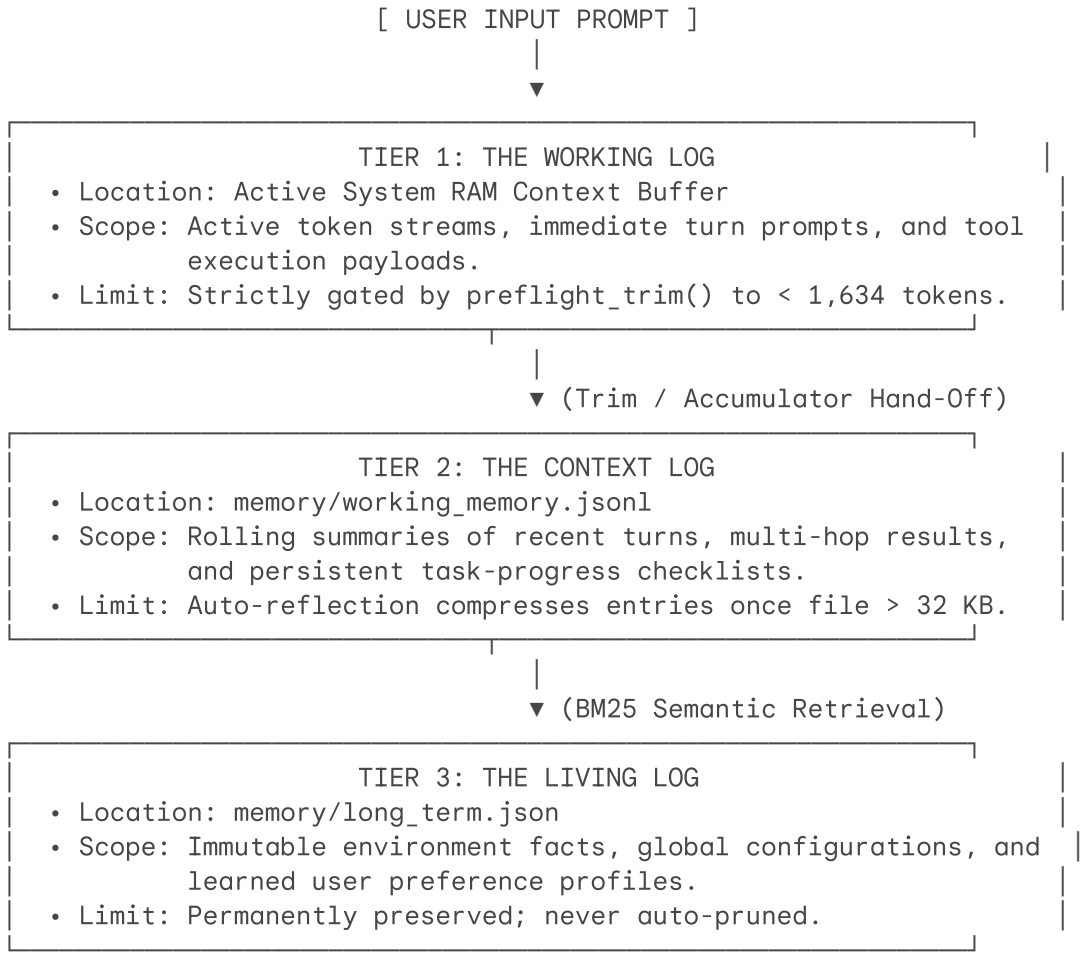
Format: Active State Representation & Schemas

Target Architecture: CPU/RAM isolated localized environments

Security Level: Fileless / RAM-Only Sensitive Segment Isolations

1. THE IN-MEMORY TIERED LIFE CYCLE

Sovereign Shards rejects disk-cached conversational footprints for sensitive pipelines. Instead, it partitions data streams into three distinct, decoupled in-memory logging tiers. This prevents VRAM saturation and limits storage write overhead on FAT32 media.



2. CHIP & STATE SERIALIZATION SCHEMAS

The `.j_chain.json` file resides in the root directory. It acts as a fault-tolerant transaction log. If the system exhausts its **3-hop execution limit** mid-task, the Python wrapper suspends execution, serializes the active state, and prompts the user with next-step directives. The chain is re-loaded automatically on the next initialization block.


```
{
  "chain_id": "shard_intake_001",
  "status": "in_progress",
  "current_turn": 2,
  "max_turns": 10,
  "last_model_used": "qwen2.5-coder-7b",
  "original_prompt": "Scan the directory app/ and refactor the exception blocks a
  "completed_steps": [
    {
      "step_id": "step_01",
      "tool": "run_tree",
      "arguments": {
        "path": "app",
        "depth": 2
      },
      "summary": "Discovered target modules: app/chat.py, app/file_tools.py, and
    }
  ],
  "key_facts": {
    "target_files": [
      "app/chat.py",
      "app/file_tools.py",
      "app/agent/tool_router.py"
    ],
    "dependencies_verified": true,
    "torvalds_compliance_violations": [
      "app/file_tools.py: Line 45 has bare 'except:'",
      "app/chat.py: Line 512 has silent pass 'except Exception: pass'"
    ]
  },
  "next_steps": "Apply run_str_replace patches to app/file_tools.py at line 45 fi
}
```


2.2 The Task Buffer Queue Schema (`memory/task_buffer.jsonl`)

For operations requiring long-horizon execution sequences, the orchestrator bypasses LLM planning drift by writing to `memory/task_buffer.jsonl` . Each entry represents an atomic step with dependency constraints.


```
{"id": "s1", "status": "completed", "instruction": "Locate files containing bare
{"id": "s2", "status": "pending", "instruction": "Apply explicit exception patch
{"id": "s3", "status": "pending", "instruction": "Apply logging exception patch t
```

2.3 The Persistent Scratch Pad Schema (`memory/scratch_pad.jsonl`)

The scratch pad is an append-only transaction log. It extracts factual details during tool runs. When a session's token buffer is trimmed, the system can rebuild its situational awareness by parsing these stored details.

```
{"timestamp": "2026-06-05T01:22:01", "source_tool": "run_search", "query": "bare
{"timestamp": "2026-06-05T01:23:45", "source_tool": "run_read", "file": "app/file
```

2.4 The Task Accumulator Structure (`memory/task_accumulator.md`)

The task accumulator aggregates massive file data across execution hops. This ensures that large data collections do not blow out the 2048-token active context window.

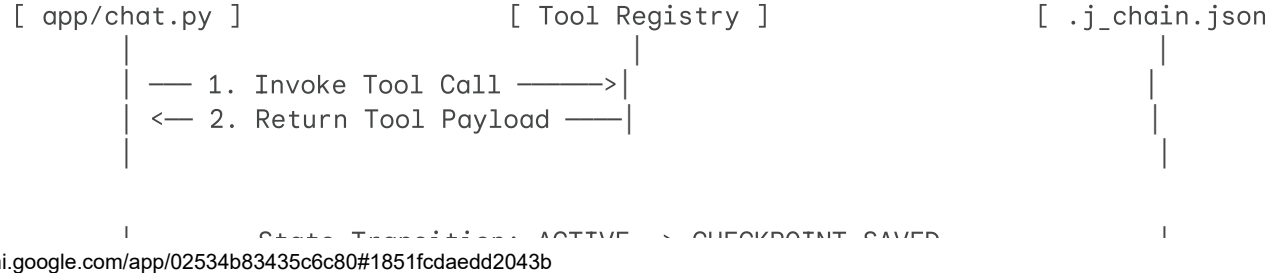
```
# SOVEREIGN SHARDS — ACTIVE ACCUMULATOR
**Active Goal:** Optimize imports across all codebase modules.
**Last Compiled Checkpoint:** 2026-06-05T01:25:00

## Compiled Data Points
* **app/chat.py**: 1,013 lines. Imports 14 stdlib packages, uses `sys`, `json`, `
* **app/file_tools.py**: 342 lines. Unused import `os` found at line 12. Imports
* **app/router.py**: 218 lines. Uses custom regex rules to intercept CLI strings.

## Unresolved Checkpoints
- [ ] Inspect import sequences inside `app/agent/tool_router.py` using `run_read`
- [ ] Remove unused dependencies from `app/file_tools.py`.
```

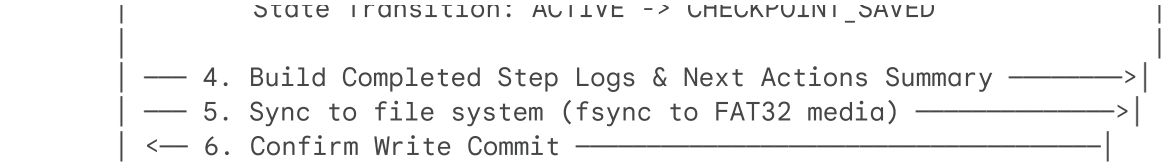
3. PROGRAMMATIC STATE TRANSITION LOGIC

These step diagrams detail how the system changes state securely. They explain how data is synced to the disk to protect files if the machine crashes or loses power.

3.1 Checkpoint Creation Sequence (Turn-Budget Exhaustion)

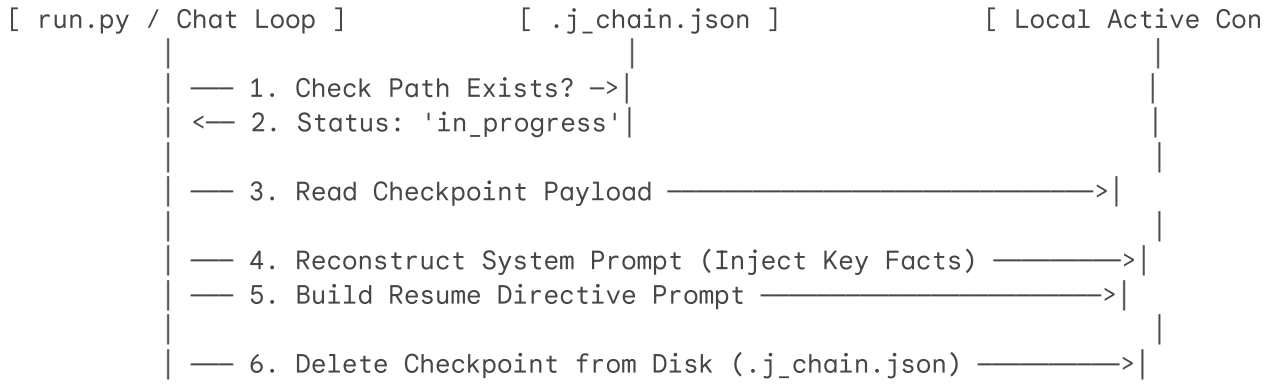
When the active execution loops consume the maximum allocated `J_TOOL_BUDGET` per turn (Default: 3), the state machine triggers a synchronous checkpoint to disk:





3.2 Checkpoint Resume Sequence (Startup Initialization)

When `run_chat` initializes, the Python host checks for an active checkpoint file before prompting the user or routing inputs:



4. METRIC AND MEMORY HEURISTICS

To maintain the absolute maximum efficiency and low physical overhead target, state managers must strictly optimize memory buffers to ensure zero disk paging:

$$\text{Active Context Window Size} = 2048 \text{ tokens}$$

$$\text{Generation Buffer Reservation} = 256 \text{ tokens}$$

$$\text{System Prompt + Tool Reference Overhead} = 680 \text{ tokens}$$

$$\text{Net Available Conversation History Budget} = 1112 \text{ tokens}$$

If active conversation history elements exceed 1112 tokens, the system executes `preflight_trim()` to shift context entries into the long-term vector maps. This keeps memory usage balanced.

```
[STATE ARCHITECTURE MANIFEST LOCKED AND PERSISTED: COMPLIANT WITH FIVE MASTERS ST
```